

Theory of Computation prep

☰ Category	studying
🕒 Created time	April 23, 2025 11:26 PM
⚙️ Status	in progress
☰ project tags	

Chapter : Intro

The Theory of Computation is a foundational area in computer science concerned with understanding the fundamental capabilities and limitations of computers. It addresses deep questions that explore the nature of computation, such as: *What can be computed? How efficiently can problems be solved? Are there problems that no algorithm can solve?*

This vast theoretical landscape is traditionally divided into three interconnected domains:

- **Automata Theory** – the study of abstract computing devices and formal languages
- **Computability Theory** – the classification of problems as solvable or unsolvable
- **Complexity Theory** – the classification of problems as computationally easy or hard

While this course focuses solely on **Automata Theory**, it is important to understand that this is only the beginning—*the tip of the iceberg*. Automata serve as simplified models of computation that allow us to reason formally about how computation works. These models are not merely academic constructs; they have direct and practical applications in areas such as compiler construction, software verification, hardware design, natural language processing, and artificial intelligence.

By starting with automata, we begin to develop a precise language and set of tools for describing computation. Concepts like **finite automata**, **regular languages**, and **context-free grammars** introduce a rigorous way to define and recognize patterns, which forms the basis for many real-world systems, especially those related to text processing and programming languages.

Although we will not formally study Computability or Complexity Theory in this course, understanding automata lays the groundwork for exploring those deeper topics in the future. Computability builds on these models to determine which problems are solvable at all, while Complexity builds on computability to ask *how efficiently* problems can be solved.

In today's world, where software systems interact with human users, external devices, and other programs in increasingly complex ways, a firm grasp of the theory behind these interactions is more valuable than ever. Automata Theory offers a way to rigorously analyze such systems and reason about their correctness and capabilities.

As you proceed through these notes, keep in mind that you are not just learning how to draw state diagrams or parse strings—you are training to understand what computation *is*, and how we can formalize and control it. This theoretical foundation will not only improve your understanding of how systems work but will also sharpen your analytical thinking and problem-solving skills across all areas of computer science.

thanks ChatGPT

Chapter 1: Languages

What is a Language?

In the context of the Theory of Computation, a **language** is a **set of strings** formed from a particular **alphabet**. It is used to describe **computational problems**. Each language represents a set of acceptable strings (solutions or valid inputs) for a given problem.

For example, the language:

$$L = \{w \in \{0, 1\}^* \mid w \text{ contains an even number of 1s}\}$$

describes the problem of recognizing all binary strings with an even number of 1s.

Key Terms

Alphabet (Σ)

A finite, non-empty set of symbols.

$$\text{Example : } \Sigma = \{a, b\}$$

String

A **finite sequence** of symbols from the alphabet.

$$L = \{w \in \{0, 1\}^* \mid w \text{ contains an even number of 1s}\}$$

Note: λ represents the **empty string**, a string of length 0.

String Operations

1. Length ($|w|$)

Number of symbols in a string.

Example: $|abba| = 4$

2. Concatenation

Appending one string to another.

$$\text{"ab"} \cdot \text{"ba"} = \text{"abba"}$$

3. Reverse

Reversing the order of symbols in a string.

$$\text{Example : } \text{rev}(\text{"abc"}) = \text{"cba"}$$

4. Power of a String

Repeated concatenation of a string with itself.

$$\text{Example : } (ab)^2 = abab$$

Language Definitions Using Σ

Let Σ be an alphabet. Then:

- Σ^* : the **Kleene star** of Σ , i.e., the set of all possible strings (including λ) over Σ
- Σ^+ : the set of all **non-empty** strings over Σ

Language

definition: Any **subset** of

$$\Sigma^*$$

So, a language

$$L \subseteq \Sigma^*$$

Example Languages

1. Language of even-length strings:

$$L = \{w \in \{a, b\}^* \mid |w| \text{ is even}\}$$

2. Language of strings with an odd number of a's:

$$L = \{w \in \{a, b\}^* \mid \text{number of } a\text{'s in } w \text{ is odd}\}$$

3. Language of binary strings ending in 01:

$$L = \{w \in \{0, 1\}^* \mid w \text{ ends in } 01\}$$

Language Operations

1. Concatenation of Languages

$$L_1 \cdot L_2 = \{xy \mid x \in L_1, y \in L_2\}$$

2. Power of a Language

$$L^n = \underbrace{L \cdot L \cdot \dots \cdot L}_{n \text{ times}}, \quad L^0 = \{\lambda\}$$

3. Kleene Star

$$\bigcup_{n=0}^{\infty} L^n$$

All strings that can be formed by concatenating zero or more strings from L

4. Kleene Plus

$$\bigcup_{n=1}^{\infty} L^n = L^* - \{\lambda\}$$

All strings formed by one or more concatenations (excludes the empty string)

5. Reverse

$$L^R = \{w^R \mid w \in L\}$$

The set of all strings in L, reversed symbol by symbol.

Chapter 2: Deterministic Finite Automata (DFA)

What is a Finite Automaton?

A **finite automaton** is a simple abstract machine used to model computation. It processes input strings symbol by symbol and moves between a finite number of states. At any given time, the machine is in one state, and based on the current input symbol, it transitions to another state (or possibly stays in the same one).

A **Deterministic Finite Automaton (DFA)** is a type of finite automaton where:

- For every state and input symbol, **there is exactly one transition** to another state.
- There is **one start state**.
- There may be one or more **accepting states** (also called final states).
- A DFA accepts a string **if it ends in an accepting state** after processing all the input.

DFA is useful for recognizing patterns, validating structured data (like identifiers in programming languages), and forms the backbone of lexical analyzers in compilers.

Formal Definition of a DFA

A DFA is formally defined as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where:

- Q is a finite set of states.
- Σ the input alphabet (a finite set of symbols).
- δ is the **transition function**: $\delta : Q \times \Sigma \rightarrow Q$.
- $q_0 \in Q$ is the **start state**.
- $F \subseteq Q$ is the set of **accepting (final) states**.

DEFINITION 1.5

A *finite automaton* is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta: Q \times \Sigma \longrightarrow Q$ is the *transition function*,¹
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.²

Examples of DFA

Example 1: DFA that accepts binary strings ending in 01

Let the input alphabet be $\Sigma = \{0,1\}$. This DFA accepts any string that ends in "01".

States:

- q_0 : Start state
- q_1 : Seen '0'
- q_2 : Seen '0' followed by '1' → accepting state

Transition function:

$$\delta(q_0, 0) = q_1$$

$$\delta(q_0, 1) = q_0$$

$$\delta(q_1, 0) = q_1$$

$$\delta(q_1, 1) = q_2$$

$$\delta(q_2, 0) = q_1$$

$$\delta(q_2, 1) = q_0$$

Start state: q_0

Final state: q_2

Regular Languages

A language is called **regular** if there exists a **finite automaton (DFA or NFA)** that accepts it.

In other words:

| A regular language is any language that can be described by a DFA.

This means that the patterns regular languages represent are computationally simple — they do not require memory beyond what a DFA can handle with its finite set of states.

Regular languages are the most basic class of languages in the **Chomsky hierarchy** and are used in areas like:

- Lexical analysis
 - Protocol design
 - Text searching
 - Pattern recognition
-

Why DFA Matters

In the big picture of computation theory, DFA is just the starting point — the **tip of the iceberg**. While DFAs are powerful enough to describe many useful problems, they **cannot handle more complex languages** that require memory, like nested parentheses or palindromes.

Understanding DFA prepares you for:

- More powerful models (like pushdown automata)
- More complex language classes (like context-free languages)
- Insights into computational limits

Chapter3: Regular Expressions and Regular Languages

What is a Regular Expression?

A **regular expression** (RE) is a formal way to describe a **regular language** — a language that can be recognized by some finite automaton (DFA or NFA).

Regular expressions are symbolic notations used to specify patterns of strings over an **alphabet** Σ .

If a language can be described using a regular expression, then it is a regular language.

Regular Expression Notation

Let a be a symbol from the alphabet Σ , and R_1, R_2 be regular expressions. The following are the basic rules and operations used to build regular expressions:

1. Single symbol

The expression a denotes the language $\{a\}$

2. Empty string

The expression ϵ (or λ) denotes the language $\{\epsilon\}$

3. Empty language

The expression $\emptyset$ denotes the empty language (no strings): $\emptyset = \{\}$

4. Union (OR)

$R_1 \cup R_2$

→ Matches strings in either R_1 or R_2

→ Written as $R_1 \mid R_2$ in many programming-style regexes

5. Concatenation

$R_1 R_2$

→ Matches strings formed by a string from R_1 followed by a string from R_2

6. Kleene Star (zero or more repetitions)

$R1^*$

→ Matches zero or more concatenations of strings from $R1$

7. Kleene Plus (one or more repetitions)

$R1^+$

→ Matches one or more concatenations of strings from $R1$

8. Grouping

Parentheses are used to group sub-expressions

→ For example: $(a \cup b)^*$ means any string of a's and b's (including the empty string)

Summary

Regular expressions are **pattern descriptors** used to generate or match **regular languages**.

They can be translated into **finite automata**, and every regular language can be described by a regular expression.

Chapter 4: Regular Expressions

1. Definition of Regular Expressions

A **regular expression (RE)** is a formal language notation used to describe **regular languages**. Regular expressions provide a way to define patterns over strings of an alphabet Σ .

A regular language is any language that can be expressed by a regular expression.

Regular expressions are **equivalent in power to finite automata (DFA/NFA)**. Every regular expression corresponds to some finite automaton, and every language accepted by a finite automaton can be described by a regular expression.

2. Basic Regular Expression Notation

Let a be any symbol from an alphabet Σ , and R_1, R_2 be regular expressions. The basic notations are:

Expression	Meaning	Example
a	Represents the language $\{a\}$	$a \rightarrow \{a\}$
ϵ (or λ)	Represents the language $\{\epsilon\}$, i.e., the empty string	$\epsilon \rightarrow \{\epsilon\}$
$\emptyset$	Represents the empty language	$\emptyset \rightarrow \{\}$
$R_1 \cup R_2$	Union — Strings in R_1 or R_2	$a \cup b = \{a, b\}$
$R_1 R_2$	Concatenation — Strings in R_1 followed by strings in R_2	$ab = \{ "ab" \}$
R_1^*	Kleene Star — Zero or more repetitions of strings in R_1	$a^* = \{ \lambda, a, aa, aaa, \dots \}$
R_1^+	Kleene Plus — One or more repetitions of strings in R_1	$a^+ = \{ a, aa, aaa, \dots \}$
(R)	Grouping to define scope of operations	$(a \cup b)^*$

3. Building Regular Expressions

Regular expressions can be **combined** to build expressions for complex languages.

Example 1:

Language of strings over $\{0,1\}$ that start with a 1:

$1(0 \cup 1)^*$

$1(0 \cup 1)^*$

Example 2:

Language of strings containing at least two a's over $\{a,b\}$:

$(a \cup b)^* a (a \cup b)^* a (a \cup b)^* (a \cup b)^* a (a \cup b)^* a (a \cup b)^*$

$(a \cup b)^* a (a \cup b)^* a (a \cup b)^*$

4. Relationship Between Regular Expressions and Regular Languages

- Every regular expression defines a **regular language**.
- Every **DFA/NFA** can be converted to an equivalent regular expression.
- Therefore, **the class of regular languages is exactly the set of languages that can be described by regular expressions.**

This equivalence makes **regular expressions** fundamental in:

- Compiler design (lexical analysis)
- Text processing tools (e.g., grep, sed, regex in programming languages)
- Pattern matching in search engines and databases

5. Closure Properties of Regular Languages

Regular languages (and therefore regular expressions) are **closed under**:

- Union
- Concatenation
- Kleene star
- Intersection
- Complementation
- Difference
- Reversal

This means that applying these operations to regular languages produces regular languages.

Summary

- Regular expressions provide a **concise, algebraic way** of describing regular languages.
- They are computationally equivalent to **finite automata**.
- Knowing regular expressions is **crucial for formal languages, programming, and algorithmic theory**.

Chapter 5: Context-Free Grammars (CFGs)

1. What is a Grammar?

A **grammar** is a set of rules that describe how to form strings in a language. Just like we have grammar rules in natural languages (like English), formal grammars define how symbols and strings can be combined in computer science and computation theory.

2. What is a Context-Free Grammar (CFG)?

A **Context-Free Grammar (CFG)** is a type of formal grammar used to generate **context-free languages**. These are languages that can't always be described by regular expressions or finite automata, especially when the structure is nested or recursive.

CFGs are important in:

- Programming languages
 - Compilers
 - Natural language processing
 - Syntax checking
-

3. Formal Definition of a CFG

A CFG is defined by 4 components:

$$G = (V, \Sigma, P, S)$$

Where:

- V = a finite set of **variables** (also called non-terminals)
- Σ = a finite set of **terminal symbols** (the alphabet of the language)
- P = a finite set of **productions (rules)** of the form:

Variable $\rightarrow$ string of variables and terminals

- S = the **start variable** (a special variable from V that starts the derivation)

4. Example of a CFG

Example grammar:

$S \rightarrow aSb \mid \lambda$

This grammar generates strings like:

λ
ab
aabb
aaabbb
aaaabbbb
...

Language of this grammar:

$L = \{ a^n b^n \mid n \geq 0 \}$

Explanation: for every 'a' added, a matching 'b' is added later.

5. Generating Strings (Derivations)

We start with the **start variable (S)** and apply productions to derive strings.

Example derivation of "aabb":

```
S → aSb
  → aaSbb
  → aaλbb
  → aabb
```

6. Language of a Grammar

The **language generated by a CFG G**, written as $L(G)$, is:

$$L(G) = \{ w \in \Sigma^* \mid S \Rightarrow^* w \}$$

Where:

- w is a string of terminal symbols
- $S \Rightarrow^* w$ means we can derive w from S using the grammar rules

7. Examples of Languages Generated by CFGs

1. Balanced parentheses:

Grammar:

$$S \rightarrow (S) \mid SS \mid \lambda$$

Example strings:

λ

$()$

$()()$

$(())$

$(())()$

...

2. Palindromes over $\{a, b\}$:

Grammar:

$$S \rightarrow aSa \mid bSb \mid a \mid b \mid \lambda$$

Example strings:

λ

a

b

aa

aba

abba

aabaa

...

8. Context-Free Languages

A **context-free language (CFL)** is any language that can be generated by a CFG.

Every regular language is also a context-free language, but not every context-free language is regular.

9. Why CFGs Are Important

CFGs are essential in:

- Defining the syntax of programming languages
 - Parsing expressions and programming code
 - Recognizing languages that regular expressions or finite automata cannot handle (e.g., nested structures)
-

10. CFGs vs Regular Languages

- **Regular languages** = described by regular expressions or DFAs → good for simple patterns

- **Context-free languages** = described by CFGs → good for complex, recursive, or nested structures
-